Bộ câu hỏi ôn bảo vệ — Vườn Hoa AI

Tài liệu này là dàn ý để từng thành viên tự diễn đạt và luyện trả lời. Các số liệu production mới nhất được ghi theo smoke test một lượt mỗi route trên Render Live ngày 23/10/2026; đây không phải benchmark thống kê hay đánh giá độ chính xác tổng quát.

1. Hãy giới thiệu kiến trúc hệ thống

Gợi ý trả lời: Frontend React/Vite chạy trên Vercel. Same-origin rewrites đưa auth/account tới Node.js/Express 5 gateway và route AI tới FastAPI Docker trên Render. Gateway hỗ trợ Neon trực tiếp khi có `DATABASE_URL` riêng; production account flow hiện proxy tới FastAPI Render, nơi health check xác nhận PostgreSQL. Bốn mô-đun AI nạp trễ theo nhu cầu; local/Compose mặc định dùng SQLite.

Mở xem: `web/src/App.jsx`, `api/main.py`, `api/account_store.py`, `compose.yaml`.

2. Bốn chức năng AI là gì, đầu vào và đầu ra ra sao?

Gợi ý trả lời: Phân loại nhận ảnh và trả top nhãn cùng điểm tin cậy; phát hiện nhận ảnh và trả hộp tọa độ, nhãn, điểm số cùng ảnh đã vẽ; tìm kiếm nhận mô tả tiếng Việt hoặc ảnh để trả ảnh hoa phù hợp; chatbot nhận câu hỏi và trả lời dựa trên đoạn kiến thức được truy xuất, kèm nguồn. API tương ứng là `/api/classify`, `/api/detect`, `/api/search/text` hoặc `/api/search/image`, và `/api/chat` hoặc `/api/chat/sync`.

Mở xem: `api/main.py`, `core/classifier.py`, `core/detector.py`, `core/retrieval.py`, `core/llm.py`.

3. Vì sao chọn các mô hình/cách làm này?

Gợi ý trả lời: ResNet-18 dùng trọng số ImageNet rồi thay lớp cuối theo 103 nhãn; cùng model tạo đặc trưng để tìm ảnh trên Render Free. YOLO11n nhận biết bốn lớp. Benchmark riêng còn có MobileCLIP2-S0 + FAISS, nhưng production Free hiện dùng đặc trưng ResNet để giảm RAM. RAG dùng BM25; Gemini Free Tier sinh câu trả lời, có fallback RAG khi provider lỗi/quota.

4. Vì sao các model không được nạp cùng lúc? Đoạn code xử lý ra sao?

Gợi ý trả lời: Render Free có ít RAM so với tổng bộ nhớ các model. `_run_heavy_model` trong `api/main.py` dùng `HEAVY_MODEL_LOCK` để chỉ chạy một tác vụ nặng tại một thời điểm, gọi `_evict_for_render_free(name)` để nhả model không cần rồi mới gọi `_require(name)` nạp model cần. Khi nhả model, API chạy garbage collection và thử `malloc_trim`. Đổi lại, lần gọi sau có thể cold start lâu; health vẫn báo route đã bật và `model_states` cho biết model nào đang cư trú.

Mở xem: `api/main.py`, hàm `_run_heavy_model` và `_evict_for_render_free`.

Mở xem: `core/classifier.py`, `core/detector.py`, `core/retrieval.py`, `core/retrieval_light.py`, `core/llm.py`.

5. Chatbot tạo câu trả lời thế nào? Nếu Gemini lỗi thì sao?

Gợi ý trả lời: Chatbot chia tài liệu thành đoạn, xếp hạng theo BM25, chọn đoạn liên quan rồi gửi ngữ cảnh cho Gemini để sinh câu trả lời có nguồn. Nếu thiếu khóa, bị giới hạn tốc độ hoặc dịch vụ lỗi, ứng dụng trả lời dựa trên phần truy xuất và báo giới hạn thay vì khẳng định đã nhận phản hồi Gemini. Có kiểm thử cho nhánh không khóa, lỗi/rate limit và câu hỏi thiếu dữ liệu.

Mở xem: `core/llm.py`; `tests/test_retriever.py`.

6. Làm sao hạn chế chatbot bịa thông tin?

Gợi ý trả lời: RAG cung cấp đoạn nguồn trước phần sinh; phản hồi trả kèm nguồn để đối chiếu. Khi không có đoạn phù hợp, hệ thống có thể báo kho kiến thức chưa có câu trả lời. Đây là biện pháp giảm rủi ro, không đảm bảo chatbot không bao giờ sai; người dùng vẫn cần kiểm tra nguồn.

Mở xem: `core/llm.py`; kiểm thử về nguồn và fallback trong `tests/test_retriever.py`.

7. Đăng nhập và dữ liệu cá nhân được xử lý ra sao?

Gợi ý trả lời: Mật khẩu được băm bằng `scrypt` với salt ngẫu nhiên. Server chỉ lưu hash của session token trong cơ sở dữ liệu, gửi token qua cookie `HttpOnly`, `SameSite=Lax`, bật `Secure` qua HTTPS và hết hạn sau 14 ngày. Sổ hoa và sổ quà được truy xuất theo user đã đăng nhập; endpoint chỉ chấp nhận khóa dữ liệu cho phép và giới hạn payload ở 256 KB. Thao tác thay đổi yêu cầu cùng origin.

Mở xem: `api/account_store.py`, `api/main.py` (đăng ký/đăng nhập/session và account data); `tests/test_account_api.py`.

8. Dữ liệu nằm ở đâu? PostgreSQL và SQLite khác nhau thế nào?

Gợi ý trả lời: Production account API dùng PostgreSQL (Neon) qua FastAPI Render. Express gateway có tùy chọn dùng Neon trực tiếp khi Vercel được cấu hình bằng `DATABASE_URL`; nếu không, nó chuyển tiếp route tài khoản tới Render. Smoke test qua Vercel same-origin đã xác nhận login, đọc sổ, logout và 401 khi chưa đăng nhập, nhưng không xác định adapter DB nào xử lý request. Local/Compose dùng SQLite, với named volume để dữ liệu còn sau khi tạo lại container. Không đưa chuỗi kết nối hay khóa API vào mã nguồn.

Mở xem: `api/account_store.py`, `compose.yaml`, `.env.example`.

9. Đã kiểm thử những gì và CI làm gì?

Gợi ý trả lời: GitHub Actions chạy pytest với PostgreSQL service để kiểm tra API/lưu trữ tài khoản; job web cài dependency theo lockfile, chạy test frontend và build production. Test backend gồm validation, ảnh lỗi, classifier/detector/search/chat, quyền riêng tư tài khoản và vòng lưu trữ PostgreSQL tùy chọn. CI xanh chứng minh các kiểm thử đã định nghĩa chạy qua; nó không tự chứng minh độ chính xác AI trên mọi trường hợp thực tế.

Mở xem: `.github/workflows/test.yml`, `tests/test_api.py`, `tests/test_account_api.py`, `tests/test_retriever.py`, `web/package.json`.

10. Nếu API trả 503 hoặc deploy vừa khởi động thì xử lý thế nào?

Gợi ý trả lời: Kiểm tra `/api/health` để xem revision, trạng thái service và model; kiểm tra `/api/startup` để xem service sẵn sàng chưa; xem Render Logs nếu readiness chưa đạt. Trên Render Free, model có thể được giải phóng sau request để tiết kiệm RAM rồi nạp lại khi cần, nên request đầu hoặc sau thời gian ngủ có thể chậm. Nếu model/data/index không hợp lệ, kiểm tra lỗi và README.

Mở xem: `api/main.py` (health, startup, nạp model/eviction); Render Logs; `README.md`.

11. Đã đo hiệu năng AI production ra sao?

Gợi ý trả lời: Smoke test production mới nhất lúc 17:45 ngày 23/10/2026 chạy qua Vercel same-origin trên đúng Render Live revision `ec114f731cd1`. Kết quả: classify HTTP 200, top-1 sunflowers, 28,94 giây; detect HTTP 200, ba vùng sunflowers, 32,92 giây; tìm ảnh HTTP 200, 5/5 kết quả sunflowers, 31,96 giây; tìm mô tả tiếng Việt HTTP 200, 5/5 sunflowers, 0,25 giây; chat HTTP 200, có 3 nguồn, 5,82 giây. Render Logs xác nhận lượt chat lúc 17:45:30 gọi Gemini Flash và Google trả HTTP 200. Mỗi route chỉ được gọi một lượt, nên đây là smoke test chứ không phải benchmark, p50/p95 hay đánh giá độ chính xác tổng quát. Thời gian ảnh chịu ảnh hưởng cold start/model eviction trên Render Free. Khi demo nên chuẩn bị ảnh hợp lệ và video dự phòng.

Lưu ý: Không trình bày điểm tin cậy của một ảnh như độ chính xác mô hình. Không nói đã benchmark nhiều lượt nếu chưa thực hiện.

12. Khởi chạy toàn bộ ứng dụng nhanh nhất thế nào?

Gợi ý trả lời: Cài Docker Desktop/Engine đang hoạt động, tạo `.env` từ `.env.example` nếu muốn bật tích hợp ngoài, rồi chạy `docker compose -f compose.yaml up --build`. Giao diện và API được phục vụ cùng container ở cổng 7860; named volume giữ dữ liệu SQLite. Dừng bằng `docker compose -f compose.yaml down`.

Mở xem: `README.md`, `compose.yaml`, `Dockerfile.compose`.

13. Nêu giới hạn hiện tại và hướng nâng cấp

Gợi ý trả lời: Render Free có RAM/CPU hạn chế, cold start và thời gian suy luận dài; việc luân phiên model là đánh đổi để các chức năng vẫn gọi được. Retrieval phụ thuộc chất lượng ảnh/index và từ vựng; chatbot bị giới hạn bởi độ phủ, chất lượng tài liệu và khả dụng Gemini. Hướng nâng cấp là dùng instance phù hợp, benchmark trên nhiều ảnh có nhãn, đo p50/p95, mở rộng kho kiến thức và theo dõi lỗi production.

Bài tập đổi mã nhanh trong 3 phút

Trước buổi bảo vệ, mỗi người chọn một thay đổi trong phần code mình thật sự hiểu và tập hoàn thành trong 3 phút. Không sửa cấu hình deploy, bí mật, hay dữ liệu tài khoản thật khi thi.

Khu vực	Thay đổi gợi ý	Kiểm tra ngay
Giao diện	Đổi placeholder tìm hoa trong `web/src/features/Search.jsx`, giữ `aria-label` có ý nghĩa	`cd web; pnpm test; pnpm run build`; mở trang tìm hoa để kiểm tra chữ mới
API	Thêm một trường hợp validation cho `TextQuery` hoặc chỉnh giới hạn `k` trong `api/main.py`	Thêm/điều chỉnh test ở `tests/test_api.py`, rồi chạy `pytest -q tests/test_api.py -k search_text`
AI tìm kiếm	Thêm một dấu hiệu hình thái vào `_visual_search_hint` trong `core/retrieval.py`	Thêm một assert trong `tests/test_retriever.py`, rồi chạy `pytest -q tests/test_retriever.py -k visual_search_hint`
Tài khoản/CSDL	Bổ sung một test quyền hoặc dữ liệu biên trong `tests/test_account_api.py` và giải thích kiểm tra quyền nằm phía server	`pytest -q tests/test_account_api.py -k private`

Trong phần hỏi đáp, trước khi gõ hãy nói rõ file và luồng liên quan; sửa một điều nhỏ; chạy đúng lệnh kiểm tra; giải thích trường hợp biên và kết quả. Dùng máy đã cài dependencies, mở sẵn repo/test, không cần bật mô hình AI nặng cho bài tập này. Mỗi thành viên tự luyện một phương án theo phần việc thật của mình.

Nhắc trước buổi bảo vệ

- Mỗi người tự chọn ít nhất một chức năng hoặc file mình đã làm và tập giải thích luồng dữ liệu.
- Các con số inference trong báo cáo là một lần đo live; nói rõ phạm vi này khi được hỏi.
- Bảng phân công có tỷ lệ và phần việc đề xuất; nhóm đối chiếu với đóng góp thực tế trước khi ký. Các ô chữ ký vẫn để trống cho từng thành viên tự ký.